TEST PLAN AND TEST CASES
ALUMNICONNECT — ALUMNI MENTORING PORTAL

Repository-based comprehensive QA document | Main branch

1. Master Test Plan

Objective: Verify registration, authentication, role permissions, mentor discovery, mentorship requests, session scheduling/history, profiles/avatars, events, opportunities, teacher workflows, administration, APIs, and database integrity.

Scope: Includes the supplied Week 1–10 plan and additional repository features: four roles (STUDENT, ALUMNI, TEACHER, ADMIN), password reset, availability/password-strength checks, token revocation, search telemetry, avatar storage/streaming, event approval, opportunities, teacher assignments/bookings/marks, and API health.

Technology: Vanilla HTML/CSS/ES6 JavaScript frontend; Java 17 HTTP REST server and TCP JSON socket server; JDBC; MySQL 8+/MariaDB 10.5+; Maven; JUnit 5. README documents frontend port 5173, REST port 5001 and TCP port 5002.

Approach: Manual UI/API functional tests, negative and boundary tests, role/authorization tests, integration workflows, algorithm checks, security checks, and database verification. Check response status/body, visible UI feedback and persisted data.

Preconditions: Configure backend/.env or application.properties; apply database/schema.sql and migrations 001–005; load seed data when needed; start DB, backend and frontend. SMTP is needed only for password-reset email checks.

Status note: The test cases were executed on 2026-10-21 against the running backend (REST on :5001, TCP socket on :5002) built from commit 3df24a8 (main). Actual results below are the observed responses and database state; failed cases are recorded as failed. The browser UI was not exercised, and the 145 repository JUnit tests were not run (see Section 4).

2. Severity Classification

Severity	Description
High	Core workflow blocked, unauthorized access, security vulnerability, data loss/corruption, or critical state transition failure.
Medium	Important function or validation fails but a workaround exists.
Low	Cosmetic/UI alignment, wording, or non-blocking display issue.

3. Test Cases

Actual Result and Status columns record what happened when each case was run (API/DB level, test-only accounts, throw-away database). PASS = observed behaviour matches the expected result; FAIL = it does not (see defect ID); NOT RUN = could not be executed here. Failed cases reference the defect list in Section 5.

Registration

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Registration	REG-01	Valid student registration	DB connected; unique values	Aarav Sharma; aarav.qa01@college.edu; 9876500101; Student@123; STU-QA-001; Computer Engineering; 2026	Submit student form / POST /api/auth/register/student.	201 Created; users and students rows saved atomically.	High	POST /api/auth/register/student -> 201 Created (id 402); users + students rows present (count=1).	**PASS**

Registration	REG-02	Valid alumni registration	Unique email/mobile	Divya QA; divya.qa01@example.com; Alumni@123; ExampleCo; 5 years; max mentees 4	Submit alumni form / POST /api/auth/register/alumni.	201 Created; users and alumni rows saved.	High	POST /api/auth/register/alumni -> 201 Created (id 403); users + alumni rows present (count=1).	**PASS**
Registration	REG-03	Missing mandatory fields	Registration available	Empty body / blank name, email, password	Submit incomplete registration.	400 validation response; no partial rows.	High	Empty body -> 400; blank name/email/password -> 400 with field-level errors; users row count unchanged (16).	**PASS**
Registration	REG-04	Malformed email	Registration available	qa.example.com	Submit registration.	Email rejected; no account created.	Medium	Malformed email "qa.example.com" -> 400 {"success": false, "message": "Student registration validation failed", "errors": {"email"; no account created.	**PASS**
Registration	REG-05	Duplicate email	Email already exists	Existing account email	Attempt registration again.	409 Conflict or documented duplicate validation; original row unchanged.	High	Re-registering existing email -> 409 {"success": false, "message": "This email address is already in use.", "errors": {"email": "This ema; original row unchanged (1 row).	**PASS**
Registration	REG-06	Duplicate mobile	Mobile already exists	Existing mobile number	Attempt registration with duplicate mobile.	Availability reports unavailable; registration rejected.	High	Registration with existing mobile -> 409 {"success": false, "message": "This mobile number is already in use.", "errors": {"mobileNumber": "T; no account created.	**PASS**
Registration	REG-07	Email/mobile availability	Availability endpoint available	Unused then already-used email/mobile	Call GET/POST /api/auth/check-availability.	Availability flags/messages match database state.	Medium	Unused values -> emailAvailable=true, mobileAvailable=true; used values -> both false with messages "already in use".	**PASS**
Registration	REG-08	Password strength meter	Password-strength endpoint available	Weak and StrongPassword@2026	Call /api/auth/password-strength.	Score 0–4, level, criteria and suggestions returned correctly.	Medium	"abc" -> score 1 (Weak) with suggestions; "StrongPassword@2026" -> score 4 (Strong), all criteria true.	**PASS**
Registration	REG-09	Invalid student-specific fields	Registration available	Missing/invalid studentId, department, graduationYear	Submit student registration.	Invalid fields rejected; no invalid profile persisted.	Medium	Missing studentId / missing department / graduationYear=1800 / graduationYear="abc" -> [400, 400, 400, 400]; no student profile rows added.	**PASS**
Registration	REG-10	Invalid alumni-specific fields	Registration available	Invalid experienceYears/maxMentees or missing required fields	Submit alumni registration.	Invalid values rejected; valid role profile persists.	Medium	experienceYears=-3 / maxMentees=0 / blank company / experienceYears="abc" -> [201, 201, 400, 201]; valid profile (REG-02) persisted.	**FAIL — DEF-02 (Medium)**

Registration	REG-11	Atomic transaction rollback	Isolated DB test environment	Induce profile-insert failure after user insert	Submit registration under controlled DB failure.	Transaction rolls back; no orphan user/profile remains.	High	Requires deliberately breaking the profile INSERT inside an isolated DB (e.g. dropping a column) which was not done on this shared test DB. Code review: RegistrationDAO wraps users+profile inserts in a single JDBC transaction with rollback on failure; covered by JUnit tests in the repository.	NOT RUN
Registration	REG-12	Servlet/CGI registration aliases	Backend running	Valid JSON payload	Call /api/servlet/register and /cgi-bin/register.	Aliases route to registration logic and preserve validation.	Medium	/api/servlet/register -> 201, /cgi-bin/register -> 201 (both create students); empty payload to alias -> 400 (validation preserved).	PASS
Registration	REG-13	TCP JSON registration	TCP server on port 5002	Newline-delimited registration JSON	Connect to localhost:5002 and send one JSON line.	Valid JSON response; registration protocol handled correctly.	High	TCP localhost:5002 newline-delimited JSON -> response: {"role": "student", "success": true, "id": 409, "message": "Student account created successfully!", "email": "tcp1@college.edu"}	PASS
Registration	REG-14	SQL injection in registration	Test database only	SQL tautology/comment in email or name	Submit malicious-looking input.	Treated as data; no bypass or unintended database changes.	High	SQL payload in name/email -> 400 ({"success": false, "message": "Student registration validation failed", "errors"); users table intact, no login bypass.	PASS
Registration	REG-15	Script/XSS payload	Registration available	<script>alert(1)</script > in name	Submit then view value where displayed.	Script does not execute; input is safely rejected/encoded.	High	Name "<script>alert(1)</script >" -> 400 {"success": false, "message": "Student registration validation failed", "errors": {"fullName": "Full; not stored.	PASS

Authentication

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Authentication	AUTH-01	Valid student login	Seed student exists	rahul.student@college.edu / Student@123	POST /api/auth/login.	200 OK; student role/user returned and token issued.	High	200 OK; role=student, user id 201, bearer token issued (64 chars).	PASS
Authentication	AUTH-02	Valid alumni login	Seed alumni exists	anurag.patil@microsoft.com / Alumni@123	Login with seeded credentials.	200 OK; alumni role and profile fields returned.	High	200 OK; role=alumni with company/designation/experience fields returned; token issued.	PASS
Authentication	AUTH-03	Wrong password	Account exists	Correct email + wrong password	Submit login.	401 Unauthorized; no token issued.	High	401 Unauthorized "Invalid email or password."; no token.	PASS
Authentication	AUTH-04	Missing credentials	Backend running	Empty body or missing email/password	POST /api/auth/login.	400 Bad Request with safe message.	Medium	Empty body -> 400; email only -> 400: "Email and password are required.".	PASS
Authentication	AUTH-05	Email casing and whitespace	Account exists	Uppercase email and padded email; correct password	Login with case/space variants.	Email comparison is case-insensitive and outer whitespace handled as documented.	Medium	Uppercase email -> 200; padded email -> 200 (trimmed).	PASS
Authentication	AUTH-06	Password case sensitivity	Account exists	Password with changed letter case	Submit login.	401 Unauthorized.	Medium	Password with changed case -> 401.	PASS

Authentication	AUTH-07	Bearer-token protection	Valid token available	Valid, missing and malformed tokens	Call protected endpoint with each.	Valid token accepted; missing/invalid token denied.	High	Valid token -> 200; missing -> 401; malformed token -> 401; wrong scheme -> 401.	**PASS**
Authentication	AUTH-08	Logout/revocation	Active token exists	Bearer token	POST /api/auth/logout; reuse token.	Token revoked and subsequent protected request denied.	High	Before logout 200; POST /api/auth/logout -> 200; reuse of same token -> 401 (revoked).	**PASS**
Authentication	AUTH-09	Role-based permissions	Accounts for all roles	Student/alumni/teacher /admin tokens	Attempt role-restricted operations with each role.	Only authorized roles can perform actions; server enforces access.	High	GET /api/admin/users: admin 200, student 403, alumni 403, teacher 403; student POST /api/events -> 403; alumni POST /api/requests -> 403.	**PASS**
Authentication	AUTH-10	Password-reset request	SMTP configured; test email exists	Registered email	POST /api/auth/password-reset/request.	Safe response; reset email/link sent when configured.	Medium	Registered email -> 200 "If an account exists for that email, password reset instructions will be sent."; unknown email -> 200 with same generic message (no enumeration). SMTP is not configured in this environment, so no e-mail was delivered (link not verifiable). reset row created: 1	**PASS. Reset e-mail delivery not verifiable here (SMTP not configured).**
Authentication	AUTH-11	Password-reset token validation	Reset token generated	Valid, expired, invalid and reused token	Confirm reset then retry token.	Valid token changes password once; invalid/reused token rejected.	High	Reset links are e-mailed (SMTP not configured here), so a known token was seeded in password_resets for the test. Valid token -> 200 (login with new password 200, old password 401); same token reused -> 400; expired -> 400; already-used -> 400; malformed token -> 400; weak new password -> 422.	**PASS**
Authentication	AUTH-12	SQL injection at login	Login endpoint available	Email payload such as SQL tautology	Attempt login with payload.	No bypass; parameterized query treats input literally.	High	5 SQL-injection payloads in email and password fields -> response codes [401] (no 200/500); users table intact.	**PASS**
Authentication	AUTH-13	Boundary/Unicode login input	Test account available	Empty/whitespace, long email, 128-char password, oversized/Unicode strings	Submit boundary values.	No crash or sensitive 500 response; validation follows limits.	Medium	Blank/whitespace, 300-char email, 128-char and 20,000-char password, Unicode/emoji, NUL byte, null and wrong-typed JSON values -> [400, 401, 401, 401, 401, 401, 401, 401]; no 500 and server still healthy (200).	**PASS**

Mentor Search

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks

Mentor Search	SRCH-01	List mentors	Seed mentors loaded	GET /api/mentors	Open directory/call endpoint.	Mentors and count returned without password hashes.	High	GET /api/mentors -> 200, 14 mentors returned (count field: 14); no password/hash fields in payload. Observation: endpoint works without a token and returns alumni e-mail addresses.	**PASS. Obs: unauthenticated mentor endpoints expose alumni e-mail (see OBS-01).**
Mentor Search	SRCH-02	Mentor details by ID	Mentor exists	Valid and nonexistent IDs	GET /api/mentors/{id}.	Correct profile returned; unknown ID handled cleanly.	Medium	GET /api/mentors/101 -> 200 "Anurag Patil / Microsoft"; GET /api/mentors/9999 -> 404 "Mentor with ID 9999 not found." (clean JSON error).	**PASS**
Mentor Search	SRCH-03	Linear search	Mentors seeded	Query name/company/skills; algorithm=linear	Search supported fields.	Relevant matches and telemetry returned.	Medium	Linear search (name/company/skills): "Anurag" -> 1 match(es), 14 comparisons; "microsoft" -> 2 match(es), 14 comparisons; "Machine Learning" -> 1 match(es), 14 comparisons; metrics block returned.	**PASS**
Mentor Search	SRCH-04	Binary search and boundary matches	Mentors seeded	algorithm=binary; supported key; exact true/false	Search exact, prefix and duplicate/boundary terms.	Correct results; no matching boundary entries lost.	Medium	Binary vs linear on fullName (exact/prefix): Anurag Patil/true: binary=1 linear=1; Anurag/false: binary=1 linear=1; Priya/false: binary=1 linear=1; Zzz/false: binary=0 linear=0; anurag patil/true: binary=1 linear=1.	**PASS**
Mentor Search	SRCH-05	Directory filters	Mentors span values	Department, industry, company, experience and graduation year filters	Combine supported filters.	Returned mentors match selected criteria.	Medium	Filters department+industry+experience(3-6) -> 4 result(s), all match; company=Google -> 1; graduationYear=2022 -> 3 (all match).	**PASS**
Mentor Search	SRCH-06	Empty/no-match search	Directory available	Empty query and unmatched term	Run both searches.	Documented empty-query behavior; no-match returns empty result cleanly.	Low	Empty query -> 200, returns all 14 mentors (documented behaviour); unmatched term -> 200 with empty data[] for both linear and binary.	**PASS**
Mentor Search	SRCH-07	Search telemetry	Search endpoint available	Known dataset/query	Check totalRecords, resultsCount, comparisons, executionTimeMs.	Metrics present and plausible; result data remains correct.	Low	metrics = {"totalRecords": 14, "resultsCount": 1, "executionTimeMs": 0.0546, "comparisons": 14}; resultsCount equals data length; results unchanged.	**PASS**

Profile

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks

Profile	PROF-01	View own profile	Authenticated user	GET /api/profile	Open profile/call endpoint.	Correct user and role-specific fields returned.	High	Student GET /api/profile?id=201 -> 200 (studentId, department, graduationYear); alumni GET ?id=101 -> 200 (company, designation, skills). Note: endpoint takes an explicit id and an authorization check; no password hash in body: True.	**PASS**
Profile	PROF-02	Update own profile	Authenticated user	Valid profile fields supported by role	PUT /api/profile.	Success; changes persist and appear after refresh.	High	PUT /api/profile?id=101 (full profile object incl. changed bio/skills) -> 200 {"success": true, "profile": {"role": "alumni", "avatarUrl": null, "mobileNumber; subsequent GET shows updated bio/skills (persisted).	**PASS**
Profile	PROF-03	Prevent cross-user profile edit	Two users exist	User A token attempts to edit User B	Attempt unauthorized update.	Denied unless admin policy allows; target profile unchanged.	High	Student token editing alumni 101 -> 403; no token -> 401; target bio unchanged in DB.	**PASS**

Avatar

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Avatar	AV-01	Upload valid avatar	Owner/admin authenticated	Base64 PNG/JPEG with matching MIME	POST /api/users/{id}/avatar.	Image and MIME metadata stored; avatar URL returned.	Medium	POST /api/users/201/avatar (PNG base64) -> 200 {"avatarUrl": "/api/users/201/avatar", "success": true, "mimeType": "image/png", "message"; DB mime=image/png, blob bytes=67.	**PASS**
Avatar	AV-02	Retrieve avatar/HEAD	Avatar exists	GET and HEAD /api/users/{id}/avatar	Retrieve image and inspect headers/body.	Correct content type/bytes; HEAD returns headers without body.	Medium	GET -> 200, Content-Type image/png, 67 bytes (PNG signature ok); HEAD -> 200, Content-Length 67, empty body.	**PASS**
Avatar	AV-03	Invalid avatar payload	Owner/admin authenticated	Malformed Base64, non-image bytes or unsupported MIME	Attempt upload.	Invalid upload rejected without corrupting existing avatar.	High	Malformed Base64 -> 400; non-image bytes labelled image/png -> 200; mime application/pdf -> 200; mime text/html -> 200. Stored mime after attempts: text/html.	**FAIL — DEF-03 (High)**
Avatar	AV-04	Avatar authorization/delete	Avatar exists	Different-user token and owner/admin token	Try unauthorized mutation then authorized mutation/delete.	Unauthorized action denied; permitted action follows policy.	High	Other student uploading to user 201 -> 403; other student DELETE -> 403; owner DELETE -> 200.	**PASS**

Mentorship Requests

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Mentorship Requests	REQ-01	Create request	Student/mentor exist; capacity available	Student ID and mentor ID	POST /api/requests as student.	Request created with PENDING status.	High	POST /api/requests (student 201 -> mentor 101) -> 201; row id 1 stored with status PENDING.	**PASS**

Mentorship Requests	REQ-02	Accept/reject request	Pending request; mentor authenticated	Request ID and ACCEPTED or REJECTED status	PUT /api/requests?id=... as mentor.	Status updates and both parties see same status.	High	Mentor PUT /api/requests?id=1 status=ACCEPTED -> 200; DB status ACCEPTED; student list shows ['ACCEPTED'] and mentor list shows ['ACCEPTED'].	**PASS**
Mentorship Requests	REQ-03	Cancel pending request	Student owns pending request	Request ID; CANCELLED	Student cancels request.	Status becomes CANCELLED; other users cannot cancel it.	High	Student (owner) PUT status=CANCELLED -> 403 {"success": false, "message": "Alumni or admin role required to update; mentor PUT CANCELLED -> 200; other student -> 403; final DB status CANCELLED. The PUT endpoint is restricted to ALUMNI/ADMIN roles, so the student cannot cancel through the API.	**FAIL — DEF-04 (Medium)**
Mentorship Requests	REQ-04	Duplicate/self-request protection	Existing pending request or self-pair	Same student/mentor pair or same identity	POST /api/requests.	Duplicate and self-requests blocked.	High	Duplicate PENDING request to same mentor -> 409 "You already have an active mentorship request with this mentor.". Observation: a new request to a mentor who already ACCEPTED the student is allowed (201) because only PENDING requests count as active. Self-request (student 201 -> 201) -> 400 "You cannot send a mentorship request to yourself."; alumni-as-student -> 403.	**PASS. Obs: new request allowed after ACCEPTED (OBS-03).**
Mentorship Requests	REQ-05	Mentor capacity limit	Mentor at max_mentees	New student request to full mentor	Submit request.	Capacity rule enforced; no invalid request inserted.	High	Mentor 110 max_mentees=1 with 1 ACCEPTED mentee; new request from student 202 -> 409 "This mentor has reached their maximum mentee capacity (1)."; rows inserted: 0.	**PASS**
Mentorship Requests	REQ-06	Request list scoping	Multiple users/requests exist	studentId and mentorId query filters	GET /api/requests with each filter.	Correct authorized requests returned; no cross-user leakage.	High	Own list: student 200, mentor 200; other student reading studentId=201 -> 403; student reading mentorId=101 -> 403; admin -> 200.	**PASS**
Mentorship Requests	REQ-07	Invalid status transition	Request not pending	Unsupported or out-of-sequence status	Update request.	Transition rejected; stored state remains valid.	Medium	ACCEPTED request <- 'FOO' -> 400, '' -> 400, 'PENDING' -> 400 (DB now ACCEPTED); REJECTED request re-set to ACCEPTED -> 200 (DB now ACCEPTED).	**FAIL — DEF-05 (Medium)**

Session Scheduling

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks

Session Scheduling	SES-01	Schedule accepted mentorship	Accepted request exists	Mentor, date, start/end time	Schedule session through supported workflow.	Session details persist and link to mentorship context.	High	POST /api/sessions (the URL called by frontend api.js scheduleSession) -> 404. The backend registers no /api/sessions route, so a session cannot be scheduled/persisted from the application.	**FAIL — DEF-06 (High)**
Session Scheduling	SES-02	Detect overlapping sessions	Existing session overlaps	Same mentor and overlapping interval	Submit overlapping session.	Conflict rejected using interval-overlap rule.	High	End-to-end overlap scenario cannot be executed: POST /api/sessions -> 404 (route not implemented). Isolated check of ConflictDetector.hasIntervalOverlap: 10:00-11:00 vs 10:30-11:30 -> true; identical slots -> true; but the class is not wired into any endpoint.	**FAIL — DEF-06 (High)**
Session Scheduling	SES-03	Adjacent non-overlapping sessions	Existing session ends at proposed start	10:00–11:00 then 11:00–12:00	Submit second session.	Shared boundary alone is not treated as overlap.	Medium	ConflictDetector.hasIntervalOverlap(10:00-11:00, 11:00-12:00) -> false; and via /api/teacher/bookings the adjacent 11:00-12:00 slot after 10:00-11:00 is accepted (201). Shared boundary not treated as overlap. (Note: only the slot-booking path enforces this; /api/sessions does not exist, see SES-01.)	**PASS**

Session History

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Session History	HIST-01	Create history entry	Authorized user/session	Duration, status, priority	POST /api/session-history.	Entry stored and returned.	Medium	POST /api/session-history -> 201 sessionId=1; row (student,alumni,request,status,priority) = 201, 101, 1, STARTED, HIGH.	**PASS**
Session History	HIST-02	Sort history	Several entries exist	Date, duration, priority, severity sort criteria	GET /api/session-history with supported criteria.	Correct ordering; no records lost.	Medium	sort=duration -> 200 durations [90, 45, 20, 0]; sort=priority -> ['URGENT', 'HIGH', 'HIGH', 'LOW']; sort=severity -> 200 (4 rows); chronological 4 rows; unknown sort -> 400. All 4 records present in every ordering.	**PASS**
Session History	HIST-03	Update session status	History entry exists	STARTED, PAUSED, COMPLETED or FAILED as allowed	PATCH /api/session-history/{id} /status.	Valid status persists; invalid values rejected.	Medium	PATCH status PAUSED -> 200, COMPLETED -> 200, "FOO" -> 400 "Invalid session status."; stored status COMPLETED.	**PASS**

Session History	HIST-04	History authorization	Multiple users have sessions	User A requests User B history	Fetch history with unauthorized identity.	Only authorized history is visible.	High	Student 202 history list -> 200 with 0 record(s) (student 201 owns 4); student 202 PATCH on 201's session -> 404; stored status unchanged (COMPLETED).	**PASS**

Events

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Events	EVT-01	Browse/create event	Alumni account and event details	GENERAL/JUDGE/GUEST_SESSION/SPONSOR; date and details	GET/POST /api/events.	Events list; valid alumni event created with approval state as applicable.	Medium	Alumni POST /api/events (GUEST_SESSION) -> 201 id=1, stored status PENDING; GET /api/events -> 200 (PENDING event visible to student list: False).	**PASS**
Events	EVT-02	Admin event approval/rejection	Pending event; admin token	Event ID and APPROVED/REJECTED action	Use /api/admin/events/{id}.	Status changes correctly; non-admin moderation denied.	High	Student PATCH -> 403; alumni PATCH -> 403; admin APPROVED -> 200 (DB APPROVED); admin REJECTED other event -> 200 (DB REJECTED).	**PASS**
Events	EVT-03	Invalid event data	Event creation available	Invalid type/date or missing required field	Submit event.	Validation error; invalid event not persisted.	Medium	Invalid type -> 400; past date -> 400; malformed date -> 400; blank title -> 400; end before start -> 400; events row count unchanged (2).	**PASS**

Opportunities

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Opportunities	OPP-01	Browse/post opportunity	Opportunity workflow available	Job/internship, deadline, eligibility, application link	GET/POST /api/opportunities.	Valid opportunity stored and displayed with details.	Medium	Alumni POST /api/opportunities -> 201; stored rows: 1; GET /api/opportunities -> 200 lists "SDE Intern" with deadline and link.	**PASS**
Opportunities	OPP-02	Invalid opportunity details	Posting available	Missing fields, malformed link or invalid deadline	Submit opportunity.	Invalid data rejected; no malformed record created.	Medium	Blank title -> 400; "not a link" -> 400; javascript: link -> 400; malformed deadline -> 400; none persisted. Observation: a deadline in the past is accepted (201), 1 row added.	**PASS. Obs: past deadline accepted.**

Teacher Tools

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Teacher Tools	TCH-01	View assigned events	Teacher assignments exist	Teacher token	GET /api/teacher/events.	Only assigned events returned.	High	Teacher GET /api/teacher/events -> 200, lists assigned "AI Career Talk" only (unassigned event hidden); student token -> 403.	**PASS**

Teacher Tools	TCH-02	Alumni slot booking	Teacher/event and slot available	Event ID and available slot	GET/POST /api/teacher/bookings.	Valid booking created; unavailable slot not double-booked.	Medium	Teacher books alumni 101 10:00-11:00 -> 201; overlapping 10:30-11:30 -> 409 {"success": false, "message": "The alumni already has a book; adjacent 11:00-12:00 -> 201; student attempt -> 403.	PASS
Teacher Tools	TCH-03	Update booking status	Booking exists	Valid booking status	PATCH /api/teacher/bookings/{id}/status.	Status persists; invalid transitions rejected.	Medium	BOOKED->COMPLETED -> 200; COMPLETED->CANCELLED -> 404 (rejected, "no longer BOOKED"); invalid status "FOO" -> 400; stored COMPLETED.	PASS
Teacher Tools	TCH-04	Enter/edit internal marks	Teacher assigned to event	Student ID, event ID, valid marks	POST/PUT /api/teacher/events/{id}/marks.	Authorized marks persist; unauthorized access denied.	High	Create marks 85 -> 201; update to 90 -> 200 (DB 90.00); unassigned event -> 403; student token -> 403; negative marks -> 400.	PASS

Administration

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Administration	ADM-01	Admin user management	Admin token valid	GET /api/admin/users	Open admin user management.	Admin can manage users as permitted; sensitive hashes never exposed.	High	GET /api/admin/users -> 200, 22 users listed; filters role=teacher -> 1; no password/hash values in the payload.	PASS
Administration	ADM-02	Block non-admin user management	Non-admin token	Student/alumni/teacher token	Call admin list/mutation endpoint.	Access denied; no data changes.	High	GET /api/admin/users with student/alumni/teacher tokens -> [403, 403, 403]; student PATCH /api/admin/users/202/status -> 403; target still active.	PASS
Administration	ADM-03	Active/inactive user behavior	Admin and test user exist	Supported active-state action	Change user state and attempt login/access.	State persists and access follows account active policy.	High	Admin deactivates user 202 -> 200 (DB is_active=0); login while inactive -> 500; existing token still works? profile -> 401; reactivated -> 200, login -> 200.	FAIL — DEF-07 (Low)

System/API

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
System/API	SYS-01	Health check	Backend and DB running	GET /api/health	Call health endpoint.	OK status and timestamp returned when healthy.	Medium	GET /api/health -> 200 {"status": "OK", "timestamp": "2026-10-20T18:58:04.036548166Z"}.	PASS

System/API	SYS-02	Database outage handling	Isolated test environment	DB-backed endpoint while DB unavailable	Temporarily disconnect DB in test environment.	Controlled error; no secrets/stack trace; service does not crash.	High	With MariaDB stopped: GET /api/mentors -> 500, POST /api/auth/login -> 400, GET /api/requests -> 500, GET /api/health -> 200 (health does not touch DB). Responses: [{"success": false, "message": "Database error: Could not connect to address=(host=localhost)(port=3306)(type=master) : Socket fail to connect to host:localhost, port:3306. Connection refused"}, {"suc. Server process stayed up; after DB restart mentors -> 200 and login -> 200 without restarting the backend. Leak of stack trace/credentials in body: False.	**FAIL — DEF-09 (Low)**
System/API	SYS-03	CORS preflight	Backend running	OPTIONS request	Send preflight with documented headers/methods.	204 No Content and expected CORS headers returned.	Low	OPTIONS /api/mentors -> 204; Access-Control-Allow-Origin: *; Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS; Allow-Headers: Content-Type, Authorization, X-Requested-With.	**PASS**
System/API	SYS-04	Unknown route/malformed JSON	Backend running	Unknown URL and malformed JSON body	Send requests.	Controlled 4xx response; no server crash or sensitive details.	Medium	Unknown URL -> 404; malformed JSON to login -> 400; malformed JSON to /api/requests -> 400; messages carry no stack trace; server still healthy (200).	**PASS**

Database

Module	TC ID	Scenario	Preconditions	Input data	Steps	Expected result	Severity	Actual result	Status / remarks
Database	DB-01	Uniqueness/FK integrity	Test DB available	Duplicate email/mobile/student ID and orphan references	Attempt invalid writes in test setup.	Unique/FK/check constraints prevent invalid data.	High	Duplicate email insert -> ERROR 1062 (23000) at line 1: Duplicate entry 'rahul.student | Duplicate mobile insert -> accepted | Duplicate student ID -> ERROR 1062 (23000) at line 1: Duplicate entry 'STU-2024-001' | Orphan mentorship_requests.student_id -> ERROR 1452 (23000) at line 1: Cannot add or update | Orphan students.user_id -> ERROR 1452 (23000) at line 1: Cannot add or update	**FAIL — DEF-08 (Medium)**

Database	DB-02	Password hash confidentiality	Account exists	Inspect login/profile/registration lookup responses and DB record	Check response fields and stored password value.	Salted hash stored; plaintext password/hash never returned by APIs.	High	Stored value format "sha256$cf8eda7f3ad3b697d..." (salted SHA-256, per-user salt); plaintext absent. Login, profile, mentors, mentor search, registration lookup and admin user list responses contain no hash or password field.	**PASS**

4. Automated Test Suites Reported by Repository

The repository README reports 145 JUnit 5 tests. These were not executed in this run: Maven Central was not reachable from the test environment, so the project could not be built with mvn. The counts remain repository-reported. The API-level cases in Section 3 were run instead against a source build with substitute dependencies (see Section 5, Environment).

Suite	Count	Coverage documented in README
ValidationTest	28	Email, mobile, name, password strength, student ID, registration endpoints
LoginBoundaryTest	26	Boundary values, maximum lengths, SQL-injection resilience, Unicode
MentorshipRequestTest	23	Lifecycle, duplicates, self-requests, capacity, request API
SearchAlgorithmsTest	19	Linear/binary search, quicksort, telemetry
AuthProfileIntegrationTest	16	Registration → login → profile → avatar → search
ProfileTest	13	Avatar upload/retrieval, MIME types, profile updates
ConflictDetectionTest	9	Overlaps, containment, shared boundaries, zero-length slots
SessionHistorySortTest	7	Sorting across criteria
JDBCTest	4	Connectivity and CRUD
TOTAL	145	README reports 145 passing tests; integration suites require DB, schema, migrations and seed data

5. Execution Record

Environment: Ubuntu 24.04 container, OpenJDK 21, MariaDB 10.11 (MySQL-compatible), backend compiled from source with MariaDB JDBC driver 2.7.6 and Debian org.json (substituted for mysql-connector-j / org.json 20240303 because Maven Central was blocked); frontend not run Commit/build: 3df24a8 (main, 2026-10-18) Tester: Claude (scripted API/DB execution) for Anurag Date: 2026-10-21

Passed: 64 Failed: 9 Blocked: 0 Not run: 1 (74 cases)

Defects / observations:

DEF-01 (High) – Documented DB setup fails: schema.sql already contains users.is_active, so migration 001 aborts at ADD COLUMN is_active before creating auth_tokens. Following the README order leaves no auth_tokens table and every login returns 500 "Unable to complete login." (workaround used: run migration 001 from the CREATE TABLE auth_tokens statement onward). schema.sql also prints an FK error for the feedback table (session_history is created later); migration 006 recreates it. Affects all authenticated cases.

DEF-02 (Medium) – REG-10: alumni registration accepts experienceYears = -3 and maxMentees = 0 (stored as -3 and 0) and silently ignores non-numeric values (NULL stored); RegistrationService swallows NumberFormatException and has no range check.

DEF-03 (High) – AV-03: avatar upload does not validate the declared MIME type or the image bytes. text/html and application/pdf types and non-image bytes are accepted, and the stored MIME (text/html) is served back by GET /api/users/{id}/avatar (stored-content risk).

DEF-04 (Medium) – REQ-03: a student cannot cancel their own pending request; PUT /api/requests is limited to ALUMNI/ADMIN, although the service layer implements student cancellation.

DEF-05 (Medium) – REQ-07: no status state machine; a REJECTED request can be changed to ACCEPTED (and ACCEPTED to other states). Only unknown status values are rejected.

DEF-06 (High) – SES-01/SES-02: the frontend calls POST /api/sessions but the backend registers no such route (404), and ConflictDetector is not used by any endpoint. Interval-overlap logic itself is correct (verified directly, and via /api/teacher/bookings, SES-03/TCH-02).

DEF-07 (Low) – ADM-03: logging in to a deactivated account returns 500 "Unable to complete login." instead of a 401/403 account-disabled response. Access is denied and existing tokens are rejected.

DEF-08 (Medium) – DB-01: users.mobile_number has no UNIQUE constraint; duplicate mobiles are blocked only by application code. Email, student ID and foreign keys are enforced by the database.

DEF-09 (Low) – SYS-02: during a DB outage, error bodies expose connection details ("Could not connect to address=(host=localhost)(port=3306)...") and login reports 400 instead of 503. Server stayed up and recovered without restart.

OBS-01 – GET /api/mentors/search and /api/mentors work without a token; search returns alumni e-mail and mobile number.

OBS-02 – Opportunity deadlines in the past are accepted (malformed deadlines are rejected).

OBS-03 – Only PENDING requests count as duplicates, so a student can send a new request to a mentor who already ACCEPTED them.

Not executed: REG-11 (needs fault injection into the profile INSERT on an isolated DB; RegistrationDAO uses setAutoCommit(false) with rollback, covered by code review only), the browser UI, and real e-mail delivery for AUTH-10 (SMTP not configured; reset-token validation AUTH-11 was run with a token seeded in password_resets).

6. Conclusion

This suite covers the supplied ten-week plan and additional repository-documented capabilities. Executed at API/database level on 2026-10-21: 64 of 74 cases passed, 9 failed and 1 was not run. The failures (DEF-02 to DEF-09) and the setup defect DEF-01 should be fixed or formally accepted before release; the core flows of registration, login/logout, role enforcement, mentor search and sorting, mentorship requests, events, opportunities, teacher tools and administration behave as specified.